

Detailed Autonomous Driving Logic

Detailed Autonomous Driving Logic

This document explains the autonomous driving logic in a layered way, from raw sensor input to final motor and steering commands. It focuses on how a practical RC car autonomy stack should work and how each decision point protects the system from bad perception or unsafe conditions.

1. The Core Idea

Autonomous driving is not one algorithm. It is a pipeline of cooperating subsystems:

- sensing
- perception
- state estimation
- planning
- control
- safety supervision
- human override

A vehicle behaves well only when these layers are connected coherently.

2. Sensors

An RC car autonomy stack commonly uses:

- camera
- LiDAR
- ultrasonic sensor
- wheel speed or hall sensor
- controller input

Each sensor answers a different question.

The camera answers:

- where is the sidewalk
- what is the visual corridor
- how centered is the car
- how confident is the visual understanding

LiDAR answers:

- where is free space
- which heading is less blocked
- how much forward clearance exists

Ultrasonic answers:

- is there an immediate close obstacle directly ahead

The wheel speed sensor answers:

- how fast the car is actually moving

The controller answers:

- whether a human wants to override autonomy

3. Perception From Camera

The camera pipeline usually begins by extracting a navigable region from the image.

There are two broad ways to do that:

- heuristic image processing
- learned segmentation model

The output should not stay as raw pixels. It should be converted into geometric quantities useful for control.

Typical camera outputs:

- left boundary of the navigable corridor
- right boundary of the navigable corridor
- center of the corridor
- heading error relative to image center
- confidence score

For sidewalk following, a segmentation mask is especially useful because it can describe width and shape, not just presence.

4. Region Of Interest

Most driving systems do not reason about the whole image equally.

They define a region of interest:

- ignore much of the sky
- ignore irrelevant upper image content
- focus on the lower-middle region where immediate driving geometry is visible

This improves:

- speed
- robustness
- noise rejection

The region of interest should match the driving problem. For a sidewalk-following RC car, the most important information is usually in the lower half to lower-middle of the frame.

5. Sidewalk Or Corridor Extraction

If the perception system gives a mask, the next step is selecting the relevant connected component.

Why this matters:

- the raw mask may contain multiple blobs
- shadows and noise may create fragments
- the car needs the component representing the drivable sidewalk corridor

A good component-selection strategy prefers regions that:

- touch the lower image area
- are near the bottom-center seed area
- are large enough to be meaningful
- look stable across rows

Once a useful component is chosen, the system can estimate:

- left edge
- right edge
- corridor center
- corridor width

6. Heading Bias

Heading bias is the simplest and most important visual control signal.

Definition:

- compare the estimated corridor center with the image center
- normalize the offset
- interpret it as "car needs to steer left or right"

If the corridor center is right of the image center:

- the desired bias is positive in one convention, or negative in another

If the corridor center is left:

- the desired bias is opposite

What matters is consistency with the steering sign used by the controller.

7. Confidence

A good autonomy stack never treats all perception outputs as equally trustworthy.

Confidence can be based on:

- model confidence
- mask stability
- corridor width plausibility
- boundary spread consistency
- perspective consistency
- agreement between sensors

Low confidence should change vehicle behavior.

Example responses to low camera confidence:

- slow down
- hold last reliable heading briefly
- rely more heavily on LiDAR
- stop if confidence collapses near an obstacle

8. LiDAR Free-Space Reasoning

LiDAR provides geometric clearance that the camera may miss.

Typical LiDAR logic:

- scan forward headings in windows
- measure minimum distance in each heading window
- assign a score to each candidate heading
- choose the best safe direction

This produces:

- best heading angle
- forward clearance estimate
- confidence or score margin

LiDAR is useful because it measures physical clearance directly, not appearance.

9. Sensor Fusion

Camera and LiDAR should not simply replace one another. They should be fused.

Fusion means:

- camera contributes path-following structure
- LiDAR contributes obstacle-aware geometry

A common fusion strategy:

- weight the camera heading by camera confidence
- weight the LiDAR heading by LiDAR confidence
- blend the two into a target heading

The exact weights matter less than the principle:

- when the camera is strong, use it more
- when the camera is weak, lean harder on LiDAR

10. Temporal Stability

Good autonomy is not only about the current frame.

Useful temporal logic includes:

- last confident heading memory
- stale-frame detection
- debounce timers for transient events
- smoothing for speed

Why:

- perception is noisy
- a single bad frame should not create a violent control jump
- short-lived ambiguity should not instantly trigger a state change

Example:

- if camera confidence falls briefly, the car can temporarily use the last confident heading rather than snapping to zero

11. Safety Layer: Immediate Obstacle Stop

Some conditions should bypass normal planning entirely.

Example:

- front ultrasonic sensor reports an obstacle closer than a fail-safe threshold

Then the logic should:

- zero target heading
- zero steering bias if needed
- command zero throttle
- apply braking logic
- record the stop reason

This is a hard safety override.

12. Safety Layer: Path Blockage

Even if the camera sees a corridor, LiDAR may show insufficient forward clearance.

If free space is below a stop threshold:

- stop
- do not continue because the visual mask alone looks reasonable

This prevents the car from driving into visible but blocked space.

13. Special Logic: Driveway Cut Detection

Driveways and curb cuts often confuse sidewalk-following systems.

Why:

- the sidewalk boundary may temporarily disappear
- the texture can become smoother and less structured
- the visual corridor may become ambiguous

A driveway-cut detector can use:

- low camera confidence
- visual texture statistics
- adequate forward LiDAR clearance
- persistence over a short time window

If these align, the system can conclude:

- the path is ambiguous in a special way
- stopping is safer than guessing

This is an example of domain-specific safety logic layered on top of perception.

14. Target Heading

After fusion and safety checks, the system chooses a target heading.

This heading is usually clamped to a maximum magnitude.

Why clamp it:

- prevents extreme commands
- keeps servo behavior predictable
- avoids unstable steering from noisy perception

Then steering command is often:

- normalized target heading divided by max heading

This produces a control value in a range like ``-1.0`` to ``1.0``.

15. Speed Policy

Autonomous throttle should depend on certainty and geometry, not be constant.

A practical three-level policy:

- cruise speed when path is clear and heading is mild
- turn speed when heading demand is large or clearance is reduced
- cautious speed when confidence is low but not low enough to stop

This couples perception quality with risk.

The more uncertain the car is, the less speed it should carry.

16. Manual Override

Human input must always dominate.

Typical override triggers:

- steering input
- throttle input
- brake input
- explicit autonomy toggle

When override occurs:

- disable autonomy immediately
- clear temporary autonomous state such as driveway-cut timers
- return full control to the human

This prevents fighting between the system and the operator.

17. Motor Command Application

After target steering and throttle are chosen, the system still should not jump instantly to final motor outputs.

It should apply rate limits:

- acceleration rate
- coasting rate
- braking rate

Why:

- protects hardware
- reduces jerk
- improves traction
- makes control behavior smoother

The same idea applies to steering deadband:

- tiny steering commands near zero can be snapped to zero
- this reduces twitching

18. Cruise Control And Autonomy Interaction

If the platform also has cruise control, autonomy and cruise logic must not conflict.

Typical rule:

- when full autonomy is active, manual cruise control is disabled
- autonomy directly chooses desired throttle

If cruise remains enabled during autonomy without clear design, the vehicle may fight itself over speed commands.

19. Logging And Debugging

A serious autonomous stack logs:

- current steering
- throttle
- brake
- target heading
- camera confidence

- LiDAR confidence
- stop reason
- forward clearance
- speed
- whether autonomy is active

This matters because most autonomy bugs are timing and interaction bugs, not just bad model outputs.

Without logs, debugging becomes guesswork.

20. Preview And Visualization

A preview window or annotated saved frames are extremely useful during testing.

Useful overlays include:

- chosen method
- confidence
- left and right boundaries
- corridor center
- target heading indication
- stop reason

Visualization makes it obvious whether the problem is:

- perception
- fusion
- speed policy
- safety logic

21. Latency Matters

An autonomous car is a real-time system.

You must care about:

- camera capture latency
- model inference latency
- control-loop frequency
- dropped frames
- stale analysis age

If perception takes too long:

- steering is based on old scenes
- the car will feel late and unstable

That is why resized inference, bounded compute, and stale-frame handling matter so much.

22. Fallback Behavior

A robust system has graceful degradation.

If the learned model fails:

- fall back to heuristic vision if available
- or reduce speed sharply
- or stop cleanly

Fallback should be explicit, not accidental.

This is especially important on small edge devices where inference may occasionally fail, stall, or miss detections.

23. Test Strategy

Autonomous driving logic should be validated in layers:

1. unit-level logic tests for geometry and thresholds
2. offline image tests
3. recorded video playback
4. live preview without motor actuation
5. low-speed controlled driving
6. wider supervised autonomous tests

This staged approach reduces the chance of unsafe surprises.

24. How The Full Loop Works

A good end-to-end loop looks like this:

1. capture the latest camera frame
2. compute camera corridor and confidence
3. read LiDAR scan and score free-space headings
4. read front ultrasonic distance
5. reject unsafe states immediately
6. detect special ambiguity cases like driveway cuts
7. fuse camera and LiDAR heading into target heading
8. choose throttle level based on confidence and turn severity
9. apply steering and motor rate limits
10. allow instant human override at any time
11. log the resulting state

Each stage exists because a single-stage "model output to motor command" system is too brittle.

25. What Usually Fails In Practice

Common real-world failure sources:

- poor camera mounting
- narrow validation split
- weak hard-case data
- inconsistent sidewalk labels
- no confidence gating
- too much speed for the perception quality
- no stale-frame protection
- no clean fallback path

Good autonomy is usually the result of disciplined engineering around these issues, not only better neural network weights.

26. Final Practical Principle

The vehicle should only drive as aggressively as the weakest reliable part of the stack allows.

If:

- perception is weak

- compute is slow
- confidence is unstable
- lighting is difficult

Then:

- slow down
- simplify behavior
- or stop

The right autonomous driving logic is not the one that looks smartest in easy scenes. It is the one that remains understandable, bounded, and safe when the scene becomes messy.